

Optimización Multiobjetivo y Algoritmos Evolutivos (MOEAs): Fundamentos e Implementación con IA

Cosijopii García García¹

¹Ingeniería en desarrollo de software y sistemas inteligentes
Universidad del Istmo Campus Ixtepec. cosijopii@unistmo.edu.mx
<https://www.cosijopii.com/>

September 22, 2026

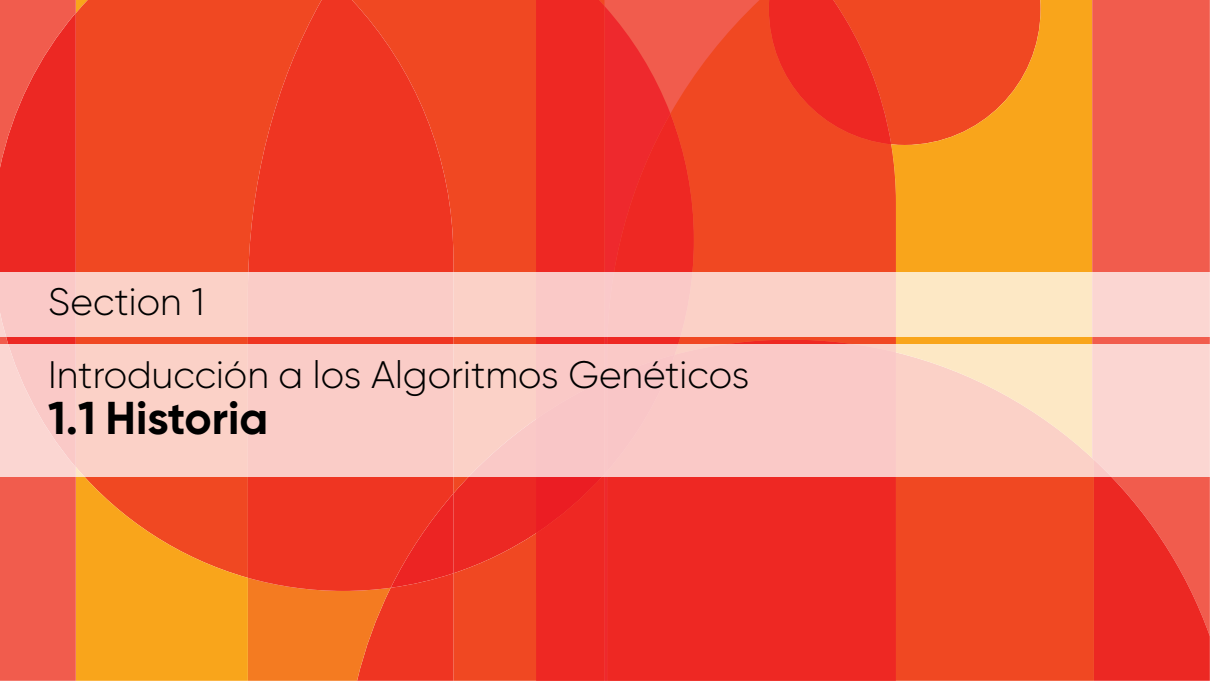
Contenido

- 1 Introducción a los Algoritmos Genéticos
 - Historia
 - El algoritmo genético y sus componentes
- 2 Algoritmos Genéticos en la Práctica
 - Problema de optimización mono -objetivo
 - El problema de la mochila
 - El Problema de Optimización Multiobjetivo



Section 1

Introducción a los Algoritmos Genéticos

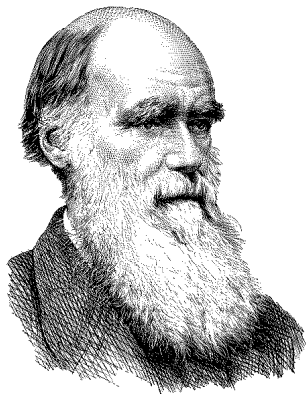


Section 1

Introducción a los Algoritmos Genéticos

1.1 Historia

- La computación evolutiva es un área de investigación dentro de las ciencias de la computación. Como su nombre indica, es un enfoque especial de la computación que se inspira en el proceso de **evolución natural**¹.
- El poder de la evolución en la naturaleza es evidente en la diversidad de especies que componen nuestro mundo, cada una adaptada para sobrevivir de manera óptima en su propio nicho.



Charles Robert Darwin

¹Eiben and Smith, *Introduction to Evolutionary Computing*, 2015.

- Ya en 1948, Turing propuso la "búsqueda genética o evolutiva".
- Bremermann realizó experimentos computacionales sobre la "optimización a través de la evolución y recombinación" en 1962².
- En la década de 1960, se desarrollaron tres implementaciones diferentes de la idea básica en distintos lugares³.

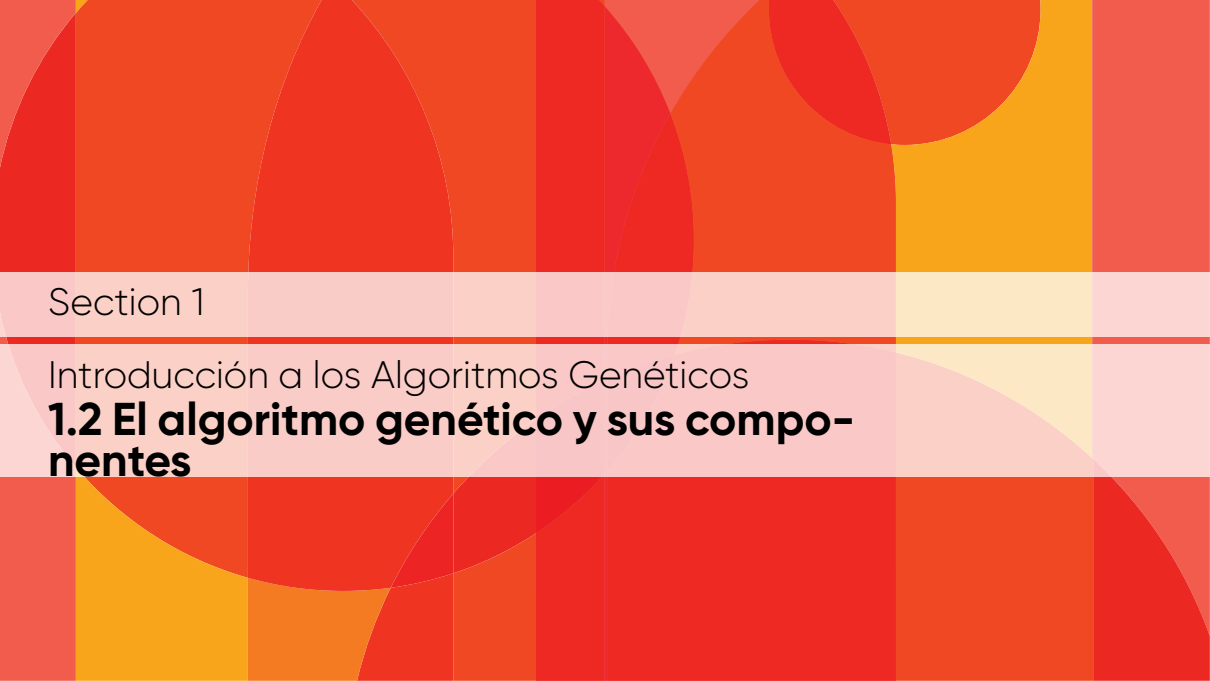
³Eiben and Smith, *Introduction to Evolutionary Computing*, 2015.

- Fogel, Owens y Walsh introdujeron la **programación evolutiva**.
- Holland llamó a su método **algoritmo genético**.
- Rechenberg y Schwefel inventaron las **estrategias evolutivas**.
- Después de 15 años, estos métodos conformaron la **Computación Evolutiva**⁴(CE).
- Finalmente, en los años 90 surgió una cuarta corriente siguiendo estas ideas generales: la **programación genética** de Koza.



John Henry Holland

⁴Eiben and Smith, *Introduction to Evolutionary Computing*, 2015.



Section 1

Introducción a los Algoritmos Genéticos

1.2 El algoritmo genético y sus componentes

Algoritmos genéticos

- El algoritmo genético (AG) es uno de los algoritmos evolutivos más populares. Su metodología de investigación se basa en la evolución natural, y fue concebido inicialmente por Holland como una forma de estudiar el comportamiento adaptativo⁵.
- El algoritmo genético canónico o simple, tiene una representación binaria, una selección proporcional a la aptitud (*fitness*), y una baja probabilidad de mutación⁶.
- Con los años, se desarrollaron nuevas características, siendo una de las más importantes el elitismo, así como diferentes tipos de recombinación y mutación.

⁵Eiben and Smith, *Introduction to Evolutionary Computing*, 2015.

⁶Sastry, Goldberg, and Kendall, *Search Methodologies*, 2014.

Algoritmo genético

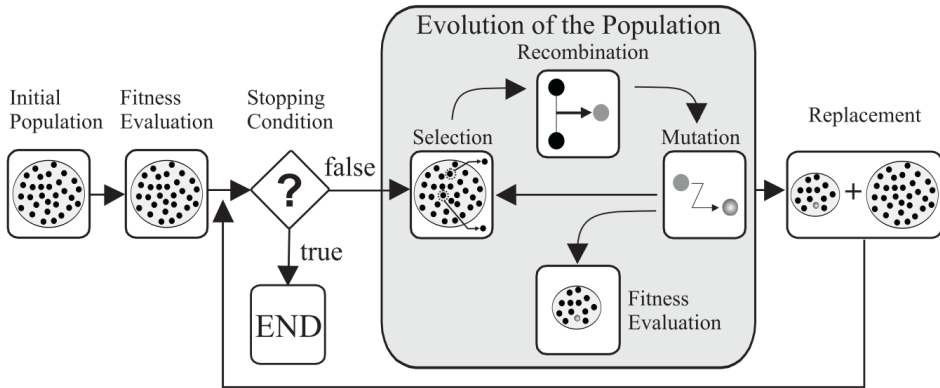


Figure 1: AG⁷

⁷Alba and Dorronsoro, *Cellular Genetic Algorithms*, 2008.

Algoritmos genéticos

- **Inicialización:** La población inicial de soluciones se genera aleatoriamente a través del espacio de búsqueda.
- **Evaluación:** Una vez creada la población, se calcula el valor de aptitud (*fitness*) de cada solución dentro de ella.
- **Selección:** En la etapa de selección, las soluciones son elegidas de acuerdo a su valor de aptitud. Existen diversas formas de procedimientos de selección.
- **Recombinación:** La información de dos o más padres se combina para crear una nueva solución que posiblemente sea mejor.
- **Mutación:** Modifica de forma local y aleatoria una solución. Implica una o más formas de añadir pequeñas perturbaciones a un individuo.
- **Reemplazo:** La descendencia creada por selección, recombinación y mutación reemplaza a la población original siguiendo algún criterio.

Representación

- Existen diferentes maneras de representar los problemas; dependiendo del tipo de representación que se utilice, los operadores evolutivos (la mutación y la recombinación o cruce) pueden variar⁸.
- Existen diferentes tipos de representación en los algoritmos evolutivos y en los algoritmos genéticos. Las representaciones más famosas son: **binaria**, **entera** y **real**.

⁸Brabazon, O'Neill, and McGarraghy, *Natural Computing Algorithms*, 2015.

Representación Binaria

- La representación binaria fue la primera en utilizarse y, históricamente, muchos algoritmos genéticos (AG) usaron esta representación independientemente del contexto del problema a resolver. Esta representación se basa en cadenas de bits que representan el genotipo. Esta cadena de bits define la longitud que tendrá y cómo se mapeará al fenotipo de la solución.

0	0	0	0	0	0	1	1	0	1
---	---	---	---	---	---	---	---	---	---

Representación Entera

- La representación entera se utiliza principalmente cuando se requiere encontrar valores óptimos para un conjunto de variables en el dominio de los números enteros. Estos valores pueden ser irrestrictos o estar restringidos a algún conjunto finito. Por ejemplo, si intentamos encontrar una ruta en una cuadrícula y estamos restringidos al conjunto $\{0,1,2,3\}$ que representa {Norte, Este, Sur, Oeste}, en este caso usar una codificación entera es mejor que una binaria. Otro ejemplo de esto puede ser la representación de redes.

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

Representación Real

- Para muchos problemas, la representación del genotipo con valores reales es la forma más natural de representarlos, y las aplicaciones de optimización actuales utilizan codificación de valores reales⁹. Esto ocurre cuando los valores de las variables provienen de una distribución continua en lugar de una distribución discreta. Un ejemplo de esto consiste en la representación de cantidades físicas como longitud, anchura, altura o peso; algunos de estos componentes pueden ser valores numéricos reales.

0.3	0.2	0.3	0.2	0.3	0.2	0.3	0.2	0.3
-----	-----	-----	-----	-----	-----	-----	-----	-----

⁹Brabazon, O'Neill, and McGarraghy, *Natural Computing Algorithms*, 2015.

Evaluación

- El papel de la función de evaluación es representar los requisitos a los que la población debe adaptarse para cumplir con el objetivo. Forma la base para la selección y, por lo tanto, facilita las mejoras. De manera más precisa, define qué significa "mejora".
- En Computación Evolutiva (CE), la función de evaluación comúnmente se llama función de aptitud (**fitness**).
- Típicamente, esta función se compone de la representación inversa (para crear el fenotipo correspondiente) seguida de una medida de calidad en el espacio del fenotipo.

Selección

- La selección es uno de los operadores principales utilizados en los algoritmos evolutivos. Su objetivo principal es encontrar las mejores soluciones dentro de una población¹⁰.
- Los criterios de selección determinan la presión de selección, que es el grado en el que se eligen las soluciones con un buen **fitness**. Si la presión de selección es demasiado **baja**, la información de los buenos padres se propagará muy lentamente por toda la población. Si la presión de selección es demasiado **alta**, la población se estancará en un óptimo local.
- Las técnicas de selección se pueden clasificar en dos categorías: **proporcional a la aptitud** y **selección ordinal**.

¹⁰Fogel, Bäck, and Michalewicz, *Evolutionary computation. Vol. 1, Basic algorithms and operators*, 2000.

Selección – Proporcional a la aptitud

- **Selección por ruleta (Roulette-wheel):** La idea principal de este método es dividir a los candidatos según su aptitud. Cuanto mayor sea la aptitud de una solución, más probable es que sea elegida.
- **Selección estocástica universal:** Es una versión ligeramente modificada de la ruleta; en lugar de usar un solo punto de selección y girar la ruleta una y otra vez hasta seleccionar a todos los individuos necesarios, giramos la rueda solo una vez y usamos múltiples puntos de selección espaciados equitativamente alrededor de la rueda. De esta forma, todos los individuos son elegidos al mismo tiempo¹¹.

¹¹Wirsansky, *Hands-on genetic algorithms with Python*.

Selección - Proporcional a la aptitud

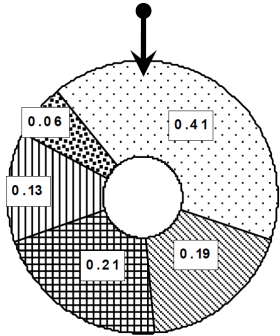


Figure 2: Selección por ruleta¹²

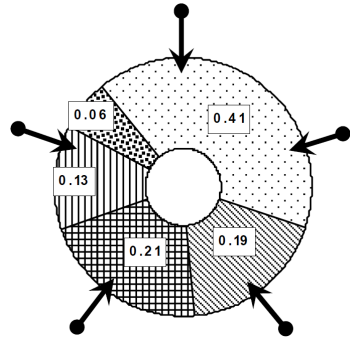


Figure 3: Selección estocástica universal

¹²Younes, Elkamel, and Areibi, "Genetic Algorithms in Chemical Engineering : A Tutorial", 2008.

Selección – Selección ordinal

- **Selección por torneo:** Se seleccionan p soluciones que entran en un torneo entre sí. El individuo con mayor aptitud en el grupo de p soluciones gana el torneo y es seleccionado como padre. El tamaño de torneo más utilizado es $p = 2$.
- **Selección por truncamiento:** En la selección por truncamiento, los individuos se ordenan de acuerdo a su valor de aptitud y se eligen los mejores en una proporción superior ($1/p$) para realizar la recombinación.

Recombinación

- La recombinación es el proceso mediante el cual se crea una nueva solución utilizando información de dos o más padres. Se considera una de las características más importantes en los algoritmos evolutivos. Una forma tradicional en la que los operadores de recombinación realizan la "cruza" es marcando subsegmentos en los genomas de los padres para luego ensamblarlos en un nuevo individuo¹³.
- La recombinación se centra en la **exploración**, intentando buscar nuevas zonas prometedoras en el espacio de búsqueda.

¹³Eiben and Smith, *Introduction to Evolutionary Computing*, 2015.

Recombinación

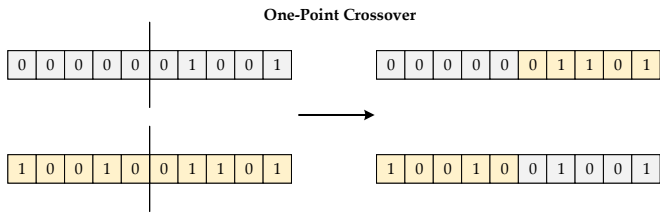


Figure 4: Cruza de un punto

Recombinación

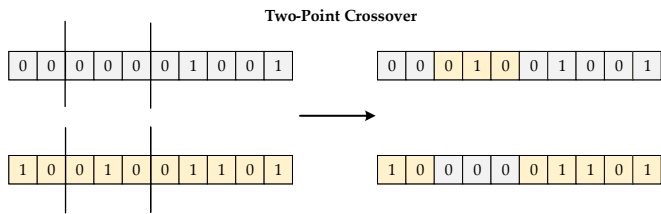


Figure 5: Cruza de dos puntos

Mutación

- La mutación es un mecanismo en el que solo interviene una solución. Las soluciones seleccionadas como padres o las soluciones de la descendencia mutan modificando ligeramente su información genética¹⁴.
- El esquema más utilizado es realizar una operación bit a bit (bitwise) en la cadena de bits con una cierta probabilidad pm .
- La mutación se centra en la **explotación**, intentando encontrar nuevas soluciones en las áreas no exploradas mediante la recombinación.

¹⁴Fogel, Bäck, and Michalewicz, *Evolutionary computation. Vol. 1, Basic algorithms and operators*, 2000.

Bitwise Mutation

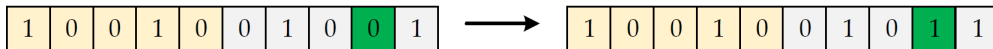


Figure 6: Operación bit a bit

Reemplazo

- El mecanismo de selección de sobrevivientes es responsable de gestionar un proceso de reducción en la población de un Algoritmo Evolutivo, pasando de un conjunto de μ padres y λ descendientes a un conjunto de μ individuos que formarán la siguiente generación.
- Hay muchas formas de hacer esto, pero una de las principales es decidir basándose en la **aptitud de los individuos**. Otra técnica se basa en la edad de la población; este método se utiliza en el SGA (Algoritmo Genético Simple), donde cada individuo existe solo por un ciclo y los padres son descartados para ser reemplazados por la descendencia. Este criterio se conoce como **modelo de población generacional**.

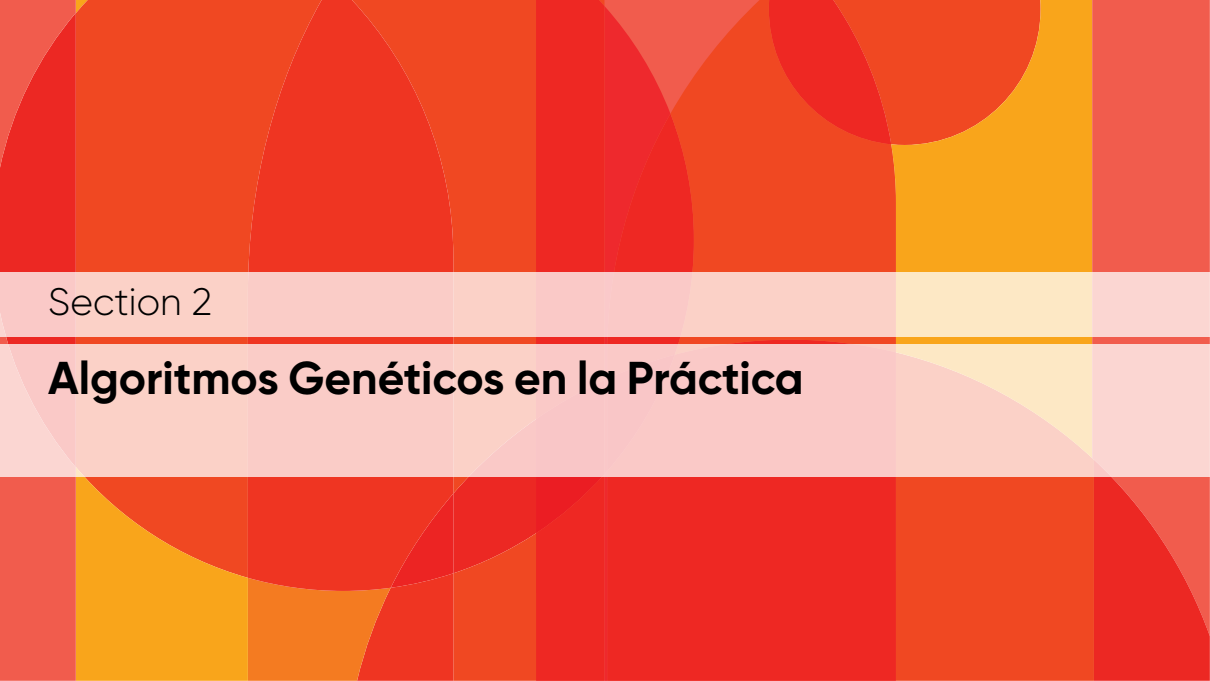
Reemplazo

- **Reemplazo del peor:** En este esquema de reemplazo se reemplazan los μ peores padres. Esto puede llevar a una convergencia prematura, cuando las soluciones se estancan en una zona limitada del espacio de búsqueda.
- **$(\mu + \lambda)$:** Proviene de las estrategias evolutivas. Se refiere al caso en el que ambos conjuntos (padres y descendencia) se clasifican dentro de un mismo grupo. Luego, las mejores μ soluciones se conservan para la siguiente generación. Esta estrategia puede verse como una generalización del criterio de reemplazar al peor.
- **Elitismo:** El elitismo se utiliza para mantener la(s) mejor(es) solución(es) en la población. Por lo tanto, si la mejor solución es elegida para ser reemplazada, y cualquier descendiente es peor o igual, la descendencia se descarta y el mejor individuo se conserva.

Algoritmo genético - Resumen

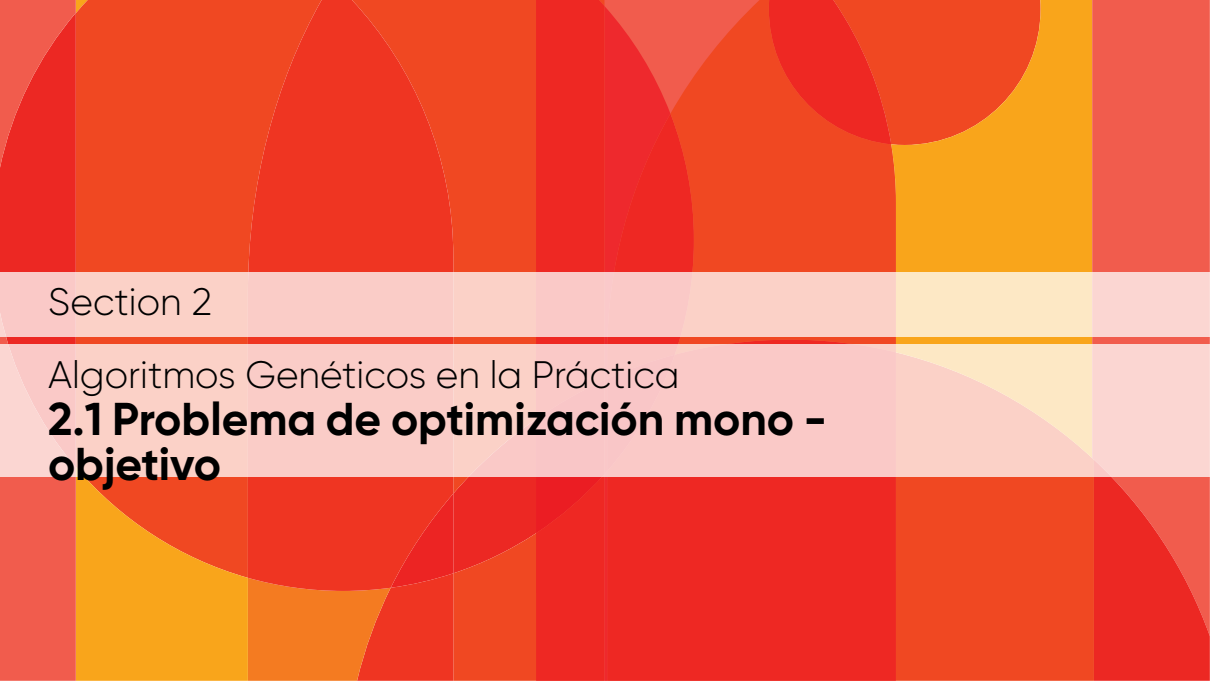
Algorithm 1: AG Simple o Canónico

```
1  $t = 0$ ;  
2 Inicializar Poblacion( $t$ );  
3 Evaluar Poblacion( $t$ );  
4 while Condición de término no satisfecha do  
5    $t = t + 1$ ;  
6   Seleccionar  $\mathbf{m}(t)$  Padres de Poblacion( $t - 1$ );  
7   Recombinar y mutar soluciones en  $\mathbf{m}(t)$ ;  
8   Crear población de descendencia  $\mathbf{m}'(t)$ ;  
9   Evaluar  $\mathbf{m}'(t)$ ;  
10  Seleccionar individuos para la siguiente generación;  
11 end while
```



Section 2

Algoritmos Genéticos en la Práctica



Section 2

Algoritmos Genéticos en la Práctica

2.1 Problema de optimización mono - objetivo

Problema de optimización mono-objetivo

- x es una **variable de decisión**. El vector $\mathbf{x}$ de $\mathbf{n}$ variables de decisión se representa mediante:

$$\mathbf{x} = [x_1, x_2, \dots, x_n]^T \quad (1)$$

- Un **problema general de optimización mono-objetivo**¹⁵ se define como minimizar (o maximizar) $f(x)$ sujeto a $g_i(x) \leq 0, i = \{1, \dots, m\}$, y $h_j(x) = 0, j = \{1, \dots, p\}, x \in \Omega$.

¹⁵Coello et al., *Evolutionary Algorithms for Solving Multi-Objective Problems*, 2007.

Problema de optimización mono-objetivo

- Las **restricciones** son impuestas por las características del entorno o los recursos y ocurren en la mayoría de los problemas de optimización. Se expresan en forma de igualdades o desigualdades matemáticas, $h_j(x) = 0, j = \{1, \dots, p\}$ y $g_i(x) \leq 0, i = \{1, \dots, m\}$. Si el número de restricciones de igualdad es mayor que el número de variables de decisión, el problema está sobrerrestringido, por lo que no hay suficientes grados de libertad para la optimización¹⁶.

¹⁶Coello et al., *Evolutionary Algorithms for Solving Multi-Objective Problems*, 2007.

Problema de optimización mono-objetivo

- El método para encontrar el **óptimo global** (que puede no ser único) de cualquier función se conoce como Optimización Global. En general, el mínimo global de un problema mono-objetivo se presenta en la siguiente definición¹⁷:

$$f: \Omega \subseteq \mathbb{R}^n \longrightarrow \mathbb{R}, \Omega \neq \emptyset, \text{ para } \mathbf{x} \in \Omega \text{ el valor } f^* \triangleq f(\mathbf{x}^*) > -\infty \quad (2)$$

- se denomina **mínimo global** si y solo si

$$\forall \mathbf{x} \in \Omega : f(\mathbf{x}^*) \leq f(\mathbf{x}) \quad (3)$$

- $\mathbf{x}^*$ es por definición la solución mínima global, f es la función objetivo, y el conjunto Ω es la región factible de $\mathbf{x}$. El objetivo de determinar la(s) solución(es) mínima(s) global(es) se denomina el problema de optimización global para un problema mono-objetivo.

¹⁷Coello et al., *Evolutionary Algorithms for Solving Multi-Objective Problems*, 2007.

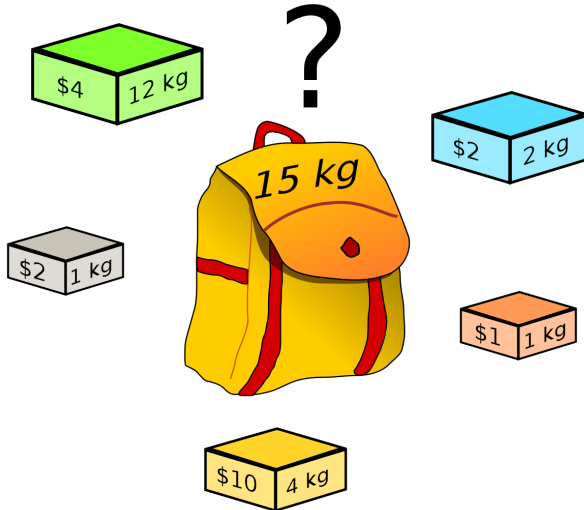


Section 2

Algoritmos Genéticos en la Práctica

2.2 El problema de la mochila

El problema de la mochila



Problema de la mochila 0-1

- Se nos da un conjunto de n artículos, cada uno con un valor v_i y un peso w_i . La tarea es seleccionar un subconjunto de esos artículos que maximice la suma de los valores, manteniendo la suma de los pesos dentro de una capacidad C_{max} .
- El objetivo es:

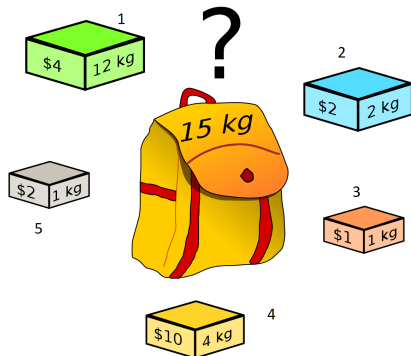
$$\text{Maximizar: } \sum_{i=1}^n v_i \cdot x_i \quad (4)$$

$$\text{Sujeto a las restricciones: } \sum_{i=1}^n w_i \cdot x_i \leq C_{max} \quad (5)$$

El problema de la mochila

Entrada: Enteros no negativos $n, v_1, \dots, v_n, w_1, \dots, w_n$ y W .

Tarea: Encontrar un subconjunto $S \subseteq \{1, \dots, n\}$ tal que $\sum_{j \in S} w_j \leq C_{max}$ y $\sum_{j \in S} v_j$ sea máximo.



Caja	V	W	X
1	$v_1 = 4$	$w_1 = 12$	$x_1 = 0$
2	$v_2 = 2$	$w_2 = 2$	$x_2 = 1$
3	$v_3 = 1$	$w_3 = 1$	$x_3 = 1$
4	$v_4 = 10$	$w_4 = 4$	$x_4 = 1$
5	$v_5 = 2$	$w_5 = 1$	$x_5 = 1$

Problema de la mochila 0-1

- Es una idea natural representar las soluciones candidatas para este problema como cadenas binarias de longitud n , donde un 1 en una posición dada indica que se incluye un artículo y un 0 que se omite.
- Al resolver un problema también es importante identificar la información que se necesita; en este caso, parece clara la función objetivo, pero también necesitamos los valores (v_i) de cada artículo así como su peso (w_i) , además de definir una capacidad máxima C_{max} .

Práctica

Abrir código en **Colab**

Section 2

Algoritmos Genéticos en la Práctica

2.3 El Problema de Optimización Multi-objetivo

Optimización multiobjetivo

- **El Problema de Optimización Multiobjetivo** se puede definir (en palabras) como el problema de encontrar¹⁸:
- “Un **vector de variables de decisión** que satisface **restricciones** y optimiza una **función vectorial** cuyos elementos representan las funciones objetivo. Estas funciones forman una descripción matemática de criterios de desempeño que generalmente están en **conflicto** entre sí. Por lo tanto, el término “optimizar” significa encontrar aquella solución que proporcione valores aceptables en todas las funciones objetivo para el tomador de decisiones.”

¹⁸ Coello et al., *Evolutionary Algorithms for Solving Multi-Objective Problems*, 2007.

Optimización multiobjetivo

$$\left. \begin{array}{l} \text{Minimizar/Maximizar } \mathbf{f}_m(\mathbf{x}), m = 1, 2, \dots, k; \\ \text{sujeto a } g_j(\mathbf{x}) \geq 0, j = 1, 2, \dots, m; \\ h_k(\mathbf{x}) = 0, k = 1, 2, \dots, p; \\ x_i^{(L)} \leq x_i \leq x_i^{(U)}, i = 1, 2, \dots, t; \end{array} \right\} \quad (6)$$

- con k objetivos, m y p son el número de restricciones de desigualdad e igualdad. Una solución $\mathbf{x} \in \mathbf{R}^n$ es un vector de n variables de decisión: $\mathbf{x} = [x_1, x_2, \dots, x_n]$, que satisface todas las restricciones y límites de las variables¹⁹.

¹⁹Coello et al., *Evolutionary Algorithms for Solving Multi-Objective Problems*, 2007.

Optimización multiobjetivo

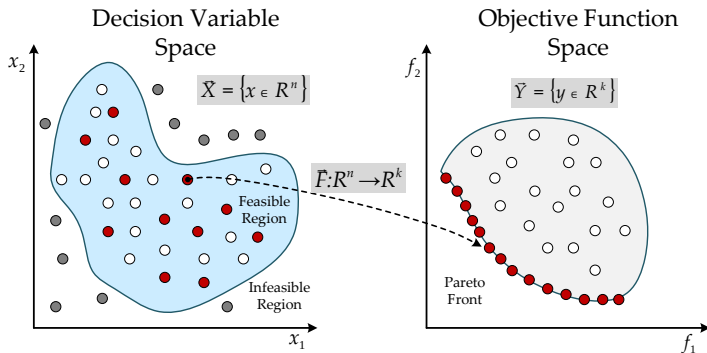


Figure 7: Mapeo del espacio de variables de decisión al espacio de funciones objetivo. Las soluciones y la zona factible están marcadas en azul. En el espacio de variables de decisión, el conjunto óptimo de Pareto está marcado con soluciones rojas y su mapeo al espacio de funciones objetivo crea el frente de Pareto.

Optimización multiobjetivo

- **Dominancia de Pareto**²⁰: Se dice que un vector $\mathbf{u} = (u_1, u_2, \dots, u_k)$ domina a otro vector $\mathbf{v} = (v_1, v_2, \dots, v_k)$ (denotado por $\mathbf{u} \preceq \mathbf{v}$) si y solo si $\mathbf{u}$ es parcialmente menor que $\mathbf{v}$, esto se especifica de la siguiente manera:
 $\forall i \in \{1, \dots, k\}, u_i \leq v_i$ y $\exists i \in \{1, \dots, k\} : u_i < v_i$.
- **Conjunto Óptimo de Pareto (POS)**: Para un MOP y $F(x)$ dados, el POS $\mathcal{P}^*$ está determinado por:

$$\mathcal{P}^* = \{\mathbf{x} \in \Omega \mid \neg \exists \mathbf{x}' \in \Omega \ F(\mathbf{x}') \preceq F(\mathbf{x})\} \quad (7)$$

- $\mathbf{v}: 5, 6$ $\mathbf{u}: 4, 6$
 $\checkmark$ $\checkmark$

- **Frente de Pareto**:
 Para un MOP dado, $F(X)$ y el POS, $\mathcal{P}^*$, el Frente de Pareto $\mathcal{PF}^*$ se puede expresar como:

$$\mathcal{PF}^* = \{\mathbf{u} = F(\mathbf{x}) \mid \mathbf{x} \in \mathcal{P}^*\} \quad (8)$$

²⁰Coello et al., *Evolutionary Algorithms for Solving Multi-Objective Problems*, 2007.

Práctica: Resolviendo un MOP en Pymoo

Abrir código en **Colab**



¿Preguntas?



Section 3

Referencias

Referencias I

- Alba, Enrique and Bernabé Dorronsoro. *Cellular Genetic Algorithms*. Vol. 42. Operations Research/Computer Science Interfaces Series. Boston, MA: Springer US, 2008. ISBN: 978-0-387-77609-5. DOI: [10.1007/978-0-387-77610-1](https://doi.org/10.1007/978-0-387-77610-1). URL: <http://link.springer.com/10.1007/978-0-387-77610-1>.
- Brabazon, Anthony, Michael O'Neill, and Seán McGarraghy. *Natural Computing Algorithms*. Natural Computing Series. Berlin, Heidelberg: Springer Berlin Heidelberg, 2015. ISBN: 978-3-662-43630-1. DOI: [10.1007/978-3-662-43631-8](https://doi.org/10.1007/978-3-662-43631-8). URL: www.springer.com/series/http://link.springer.com/10.1007/978-3-662-43631-8.
- Coello, Carlos A Coello et al. *Evolutionary Algorithms for Solving Multi-Objective Problems*. Genetic and Evolutionary Computation Series. Boston, MA: Springer US, 2007, p. 315. ISBN: 978-0-387-33254-3. DOI: [10.1007/978-0-387-36797-2](https://doi.org/10.1007/978-0-387-36797-2). URL: <http://link.springer.com/10.1007/978-0-387-36797-2>
<http://link.springer.com/10.1007/978-0-387-36797-2>.
- Eiben, A.E. and J.E. Smith. *Introduction to Evolutionary Computing*. Natural Computing Series. Berlin, Heidelberg: Springer Berlin Heidelberg, 2015. ISBN: 978-3-662-44873-1. DOI: [10.1007/978-3-662-44874-8](https://doi.org/10.1007/978-3-662-44874-8). URL: <http://link.springer.com/10.1007/978-3-662-44874-8>.

Referencias II

- Fogel, David B., Thomas Bäck, and Zbigniew. Michalewicz. *Evolutionary computation. Vol. 1, Basic algorithms and operators*. Institute of Physics Pub, 2000, p. 384. ISBN: 0750306645.
- Sastry, Sastry, David E. Goldberg, and Graham Kendall. *Search Methodologies*. Ed. by Edmund K. Burke and Graham Kendall. Boston, MA: Springer US, 2014. Chap. 4, p. 716. ISBN: 978-1-4614-6939-1. DOI: [10.1007/978-1-4614-6940-7](https://doi.org/10.1007/978-1-4614-6940-7). arXiv: [arXiv:1011.1669v3](https://arxiv.org/abs/1011.1669v3). URL: <https://link.springer.com/content/pdf/10.1007%2F978-1-4614-6940-7.pdf>[http://link.springer.com/10.1007/978-1-4614-6940-7](https://link.springer.com/10.1007/978-1-4614-6940-7).
- Wirsansky, Eyal. *Hands-on genetic algorithms with Python*. ISBN: 9781838557744.
- Younes, Abdunnaser, Ali Elkamel, and Shawki Areibi. "Genetic Algorithms in Chemical Engineering : A Tutorial". In: 2008.